

CSCE 605 COMPILER DESIGN · MODULE 07 · LECTURE NOTES

Intermediate Representation and SSA

The decision everything downstream depends on.

MODULE OUTCOMES

- Build a control flow graph from an AST.
- Compute dominators and dominance frontiers.
- Place ϕ functions and rename to construct SSA.
- Explain why SSA makes optimisation practical.
- Destruct SSA correctly, including the swap problem.

1 The control flow graph

BASIC BLOCKS, AND THE GRAPH OVER THEM

A basic block is a maximal straight-line sequence of instructions — control enters only at the first instruction and leaves only after the last. No branches in, no branches out, except at the ends.

So control flow *within* a block is entirely trivial, and all the interesting structure lives in the edges between blocks. That separation is the whole reason for the construct.

Blocks are found by identifying *leaders*: the first instruction of the function, any instruction that is a branch target, and any instruction immediately following a branch. Each leader begins a block that runs until the next leader.

The CFG makes implicit control flow explicit, which is the precondition for every dataflow analysis in Module 08 — you cannot compute what reaches a program point without first knowing what can precede it. **This is the same move as Module 01 §2's argument about representations**: the AST had the control flow too, but only implicitly, and implicit is the same as absent for the purposes of analysis.

2 Static single assignment

BEFORE	AFTER (SSA)
x = 1	x1 = 1
if (c)	if (c)
x = 2	x2 = 2
y = x + 1	x3 = phi(x1, x2)
	y1 = x3 + 1

EVERY VARIABLE ASSIGNED EXACTLY ONCE.

phi is NOT a real instruction. It selects by which predecessor we arrived from. It sits at the TOP of a block, and all phis in a block are SIMULTANEOUS, not sequential (see section 4).

WHAT IT BUYS:

"where is this defined?"	-> exactly one answer, always
def-use chains	-> explicit, no analysis
constant propagation	-> becomes local
dead code elimination	-> "no uses" means delete

SSA IS WHY MODERN OPTIMISATION IS PRACTICAL

Without SSA, answering 'what is the value of x at this point?' requires a dataflow analysis over the whole function (Module 08) — and that analysis must be redone, or carefully invalidated, every time any transformation changes anything.

With SSA it is a pointer dereference. The use names its unique definition directly, and transformations maintain that property incrementally as they go.

So analyses that were quadratic become linear, and optimisations that were too expensive to run become cheap enough to run on every function at every optimisation level. That is the difference between an optimisation existing in the literature and existing in your compiler.

LLVM, GCC, HotSpot, V8, and SPIR-V are all SSA-based. SPIR-V is SSA in its on-disk format, not merely internally — which means **every shader you have ever written was compiled through an SSA representation you can disassemble and read** (Module 13). That is an unusually direct connection between a compiler-theory idea and something already on your machine.

3 Constructing SSA

$DF(A) = \{ B : A \text{ dominates a predecessor of } B, \text{ and } A \text{ does not strictly dominate } B \}$

A dominates B when every path from the entry block to B passes through A. The **immediate dominator** $idom(B)$ is the closest strict dominator, and these form the **dominator tree**. The **dominance frontier** **$DF(A)$ is where A's dominance runs out** — the first blocks reachable from A that A does not itself dominate, which is precisely where a definition in A stops being the only one that can reach. **So that is exactly where a ϕ is needed**, and stating the frontier that way makes the placement rule obvious rather than magical.

```

PHASE 1 -- PLACE PHIS
  compute the dominator tree, then dominance frontiers
  for each variable v:
    W = { blocks assigning v }
    while W not empty:
      b = W.pop()
      for each d in DF(b):
        if d has no phi for v:
          insert phi for v at top of d
          W.add(d)          # the phi is ITSELF a definition

PHASE 2 -- RENAME
  walk the dominator tree in preorder with a stack per variable:
    DEF -> push a fresh name
    USE -> replace with stack top
    phi operand in a successor -> stack top for THIS edge
    leaving a block -> pop everything pushed in it

```

The $W.add(d)$ is the step people omit. Inserting a ϕ creates a new definition of v in block d , and that definition may itself require ϕ s at d 's dominance frontier. Without the iteration the construction terminates with missing ϕ s, the IR verifier passes if it is not checking dominance, and the wrong values are used at merge points — a miscompile of exactly the kind Module 01 §4 described.

4 Destructing SSA

ϕ IS NOT AN INSTRUCTION, SO IT MUST BE REMOVED

No machine has a ϕ instruction. Before code generation, each ϕ must be replaced by ordinary copies placed at the end of each predecessor block — the copy on edge i assigns the ϕ 's i -th operand to its result.

The naive translation is wrong. Because all the ϕ s at the top of a block execute *simultaneously*, a group of them can collectively describe a permutation of values — and emitting the copies sequentially destroys it.

The swap problem is the canonical case: $a2 = \phi(b1, \dots)$ together with $b2 = \phi(a1, \dots)$ means 'swap a and b on this edge'. Emitting $a = b$ then $b = a$ loses the original a entirely and assigns b to both.

So the copies form a *parallel copy* that must be sequentialised correctly: build the dependency graph, emit the copies that are not part of a cycle in topological order, and break each remaining cycle with one temporary. **This is a real bug that has shipped in real compilers**, it only manifests after copy propagation has made the permutation possible, and it is why SSA destruction is treated as its own carefully-tested phase rather than a mechanical rewrite.

- **Critical edges must be split.** An edge from a block with multiple successors to a block with multiple predecessors has nowhere to put the copy — putting it in the source block executes it on the wrong paths, and putting it in the destination executes it for the wrong predecessors. **Insert an empty block on the edge.** It is cheap insurance and most compilers do it unconditionally before destruction.
- **Coalescing removes most of the inserted copies** by assigning the ϕ 's operands and result the same register, which is a register allocation concern (Module 11). A naive destruction followed by good coalescing is usually better than a clever destruction.
- **The lost-copy problem** is the swap problem's sibling: aggressive copy propagation can extend a value's live range across the point where the copy was to be inserted, so the inserted copy overwrites a value still needed.
- **Some compilers remain in SSA through register allocation. SSA interference graphs are chordal**, which makes optimal colouring polynomial rather than NP-hard — a genuinely surprising result that Module 11 returns to.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Cytron, Ferrante, Rosen, Wegman & Zadeck — Efficiently Computing SSA Form (free)	The original construction of §3 , including dominance frontiers. Still the clearest statement of the placement argument.
Cooper, Harvey & Kennedy — A Simple, Fast Dominance Algorithm (free)	The iterative dominator algorithm — simpler than Lengauer–Tarjan and faster on real CFGs. Four pages; implement this one.

Resource	What it covers for this module
Briggs, Cooper, Harvey & Simpson — Practical Improvements to the Construction and Destruction of SSA Form (free)	The ϕ swap and lost-copy problems , named and solved by the people who found them.
LLVM Language Reference, and SSA Book (free)	A full free book on SSA, and the IR specification to read alongside it.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Build a CFG from your AST and render it with Graphviz.
- 2 **Implement the Cooper–Harvey–Kennedy dominator algorithm** and verify it against a brute-force definition.
- 3 Compute dominance frontiers and check them by hand on a small CFG.
- 4 **Implement ϕ placement with the worklist**, then remove the `W.add(d)` line and find a program that now gets wrong answers.
- 5 Implement renaming over the dominator tree.
- 6 **Write an IR verifier that checks dominance** — every use is dominated by its definition.
- 7 Compare your SSA against `clang -S -emit-llvm -O1` on the same source.
- 8 **Implement naive ϕ destruction and construct the swap problem**. Then fix it with cycle breaking.
- 9 Implement critical edge splitting and show a case that needs it.
- 10 **Disassemble a SPIR-V shader** and identify its ϕ instructions and basic blocks.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define a basic block and say how leaders are found.
- 2 What is SSA and what does the single-assignment property buy?
- 3 What does ϕ mean, where does it sit, and why are ϕ s simultaneous?
- 4 Define dominance, immediate dominance, and the dominance frontier.
- 5 Why is the dominance frontier the right place for a ϕ ?
- 6 Why must ϕ placement iterate?
- 7 Describe the renaming walk.

- 8 State the swap problem and its fix.
- 9 What is a critical edge and why split it?